# CRYPTOGRAPHY ISN'T JUST XOR

Presented by:

- Grinza Mattia, (griin)

# INDEX


01 Introduction to Asymmetric Cryptography

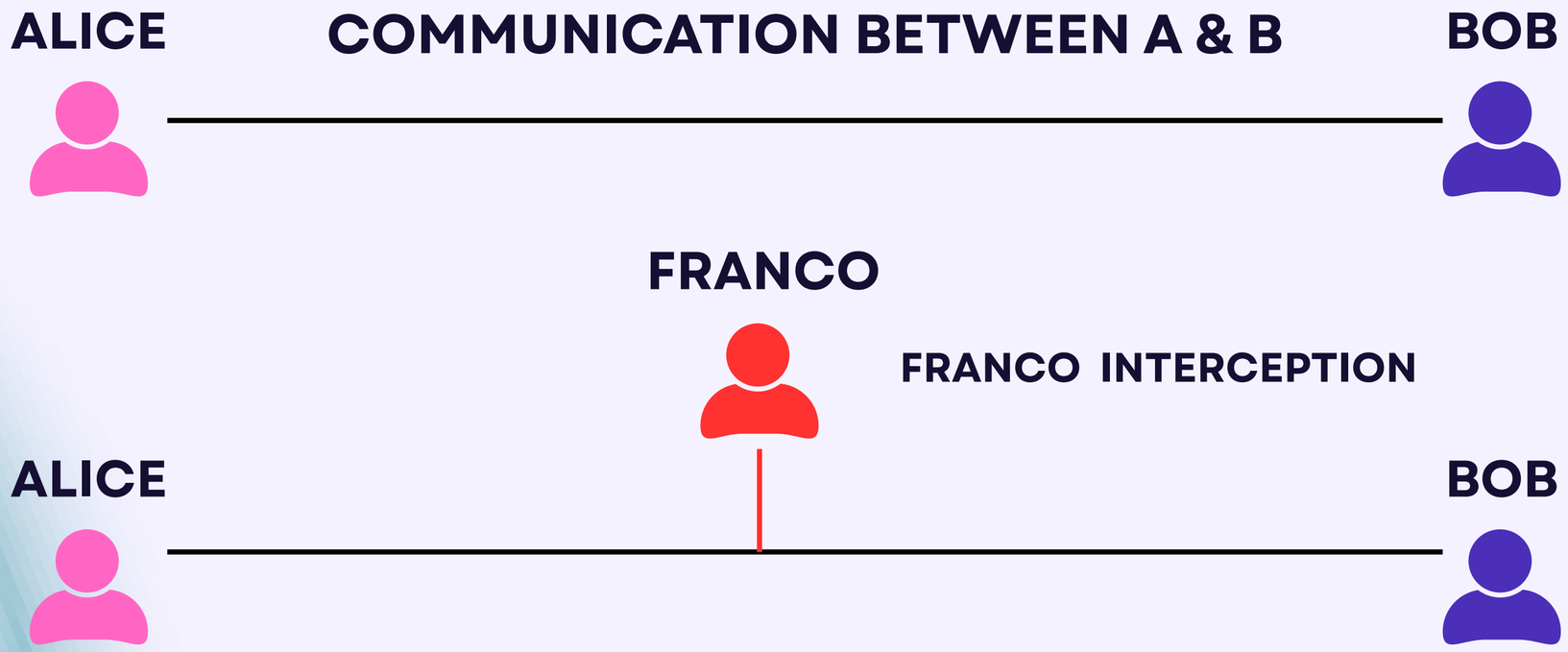
02 Diffie - Hellman Protocol

03 RSA and Digital Signatures ( DSA )

04 Introduction to SageMath (Linearity and Commitments)

# 01 Introduction to Asymmetric Cryptography

## WHY IS IT NECESSARY??



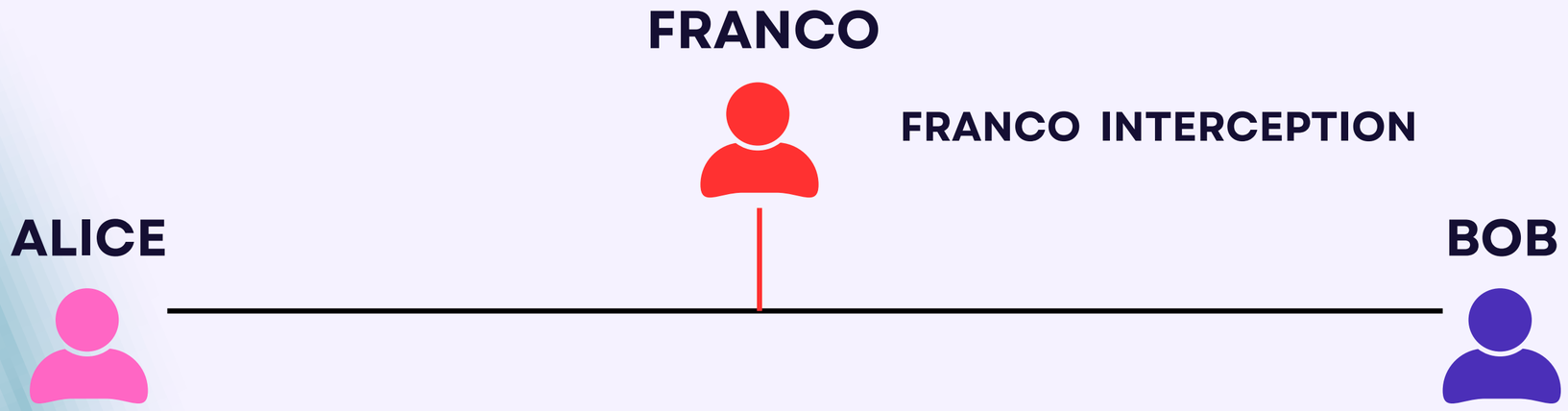

FRANCO CAN READ ALL THE COMMUNICATIONS

# 01 Introduction to Asymmetric Cryptography

## MAN IN THE MIDDLE ATTACK

### FRANCO CAN READ TRAFFIC DATA IF HE HAS ACCESS TO THE ROUTER

### HOW DOES IT WORK?




### FRANCO CAN READ ALL THE COMMUNICATIONS

# 01 Introduction to Asymmetric Cryptography

BASICALLY FRANCO IS ABLE TO CONVINCE THE TARGET (ALICE/BOB) THAT HE IS THE ROUTER AND MAKE THEM SEND THE MESSAGES THROUGH HIM

**This technique is called ARP SPOOFING**

DETAILS IN THE APPENDIX

# WHAT IS CRYPTOGRAPHY?

Cryptography is the art of making a message readable only to its intended recipient.

We have a lot of ciphers we can use to achieve this result

Every cipher needs a secret key though

**When the key is one, we call this field of study Symmetric Cryptography**

# 01 Introduction to Asymmetric Cryptography

## CERTAINTIES IN LIFE

very few

1. death

2. **Today, If two parties share a secret key, they can communicate in an extremely secure way.**

# 01 Introduction to Asymmetric Cryptography

## OUR MISSION FOR TODAY



Find a safe way to share the secret key

So even if FRANCO is listening to our conversations he can’t understand anything

# 01 Introduction to Asymmetric Cryptography

## THE IDEA

**We want to create a computationally hard problem**

### The Mathematical Foundation

**one-way function** is a mathematical operation that is easy to compute in one direction, **but computationally infeasible to invert given only the output.** In asymmetric cryptography, this guarantees that generating a public key from a private value or encrypting a message is fast, while deriving the original secret from the public data would require an impractical amount of time and computing power.

**ALICE** **COMMUNICATION BETWEEN A & B** **BOB**





**Generates a private key**
**Calculate and send the public key**

**Generates a private key**
**Calculate and send the public key**

# 01 Introduction to Asymmetric Cryptography

## SOMEBODY ELSE SOLVED THIS PROBLEM

who?

Whitfield Diffie and Martin Hellman in 1976

# DIFFIE-HELLMAN PROTOCOL

# 02 Diffie - Hellman Protocol

## FIRST WE NEED SOME MATH FUNDAMENTALS

Diffie-Hellman works in $\mathbb{Z}_p$, where p is a prime number

$$\mathbb{Z}_p = \{0, 1, \ldots, p-2, p-1\} \quad |\mathbb{Z}_p| = p$$

**Why?**

**You'll understand later**

How does math work in $\mathbb{Z}_p$

## 02 Diffie - Hellman Protocol

Every element in $\mathbb{Z}$ can be mapped in $\mathbb{Z}_p$ like this:

$$\text{map} : \mathbb{Z} \to \mathbb{Z}_p$$

$$y \equiv x \pmod{p}$$
$$\implies x \equiv y \pmod{p}$$



**Clocks are magic : they can fit 24 hours in just 12 numbers**

**If we say 15.00 or 3.00 the clock always show 3**

Why 3 ? because $15 \bmod 12 \equiv 3 \bmod 12$

If a day had 36 hours, I could still use this clock

$$27 \bmod 12 \equiv 15 \bmod 12 \equiv 3 \bmod 12$$

Examples:

$13 \bmod 3 \equiv 1, 13 = 4 \cdot 3 + 1$

$14 \bmod 5 \equiv 4, 14 = 5 \cdot 2 + 4$

$7 \bmod 10 \equiv 7, 7 = 0 \cdot 10 + 7$

$17 \bmod 10 \equiv 7, 17 = 1 \cdot 10 + 7$

$(17 \equiv 7 + p \bmod p)$

$40 \bmod 20 \equiv 0, 40 = 2 \cdot 20 + 0$

**We can clearly see that :** $x \equiv x + p \pmod{p}$

```
y = x % p
```

How does arithmetic work in $\mathbb{Z}_p$?

# 02 Diffie - Hellman Protocol

$$[(a \bmod p) + (b \bmod p)] \bmod p \equiv (a + b) \pmod{p}$$

```
y = (a+b) % p
```

$$a + 0 \bmod p = a \pmod{p}$$

$$[(a \bmod p) - (b \bmod p)] \bmod p \equiv (a - b) \pmod{p}$$

```
y = (a-b) % p
```

$$a - 0 \bmod p = a \pmod{p} \qquad -a \equiv p - a \pmod{p}$$

$$[(a \bmod p) \times (b \bmod p)] \bmod p \equiv (a \times b) \pmod{p}$$

```
y = (a*b) % p
```

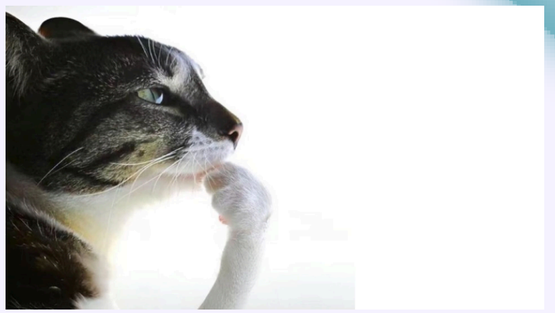
Could be useful

$$a \times 0 \bmod p \equiv 0 \pmod{p} \qquad a \times 1 \bmod p \equiv a \pmod{p}$$

What about division ??

# 02 Diffie - Hellman Protocol

## What actually is division?



$$\frac{a}{b} \equiv a \cdot \frac{1}{b}$$

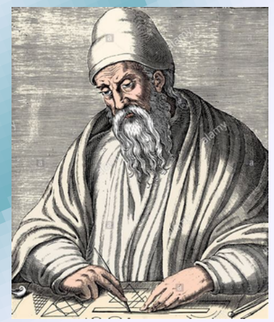
$$\frac{1}{b} \text{ is a value such that } b \cdot \frac{1}{b} = 1$$

We basically need to find $x \in \mathbb{Z}_p$ such that $b \cdot x \equiv 1 \pmod{p}$

Such an $x$ is defined as the inverse of $b$ with respect to $p$

**Does this x always exists? Is there a fast way to look for x ??**

# 02 Diffie - Hellman Protocol



**This guy can answer all of our questions**

Euclid

## The extended Euclidean Algorithm

This algorithm is able to find integer solutions to the equation $a \cdot x_0 + m \cdot x_1 = \gcd(a, m)$

Which basically means we can find the solution to $a \cdot x_0 \equiv \gcd(a, m) \pmod{m}$

if $\gcd(a, m) = 1 \implies$ We can guarantee that the inverse exists

```
x = pow(a,-1,m)
```

What about finding solutions to $a \cdot x \equiv b \pmod{p}$ for an abitrary b

$x$ exists only if $\gcd(a, m)$ divides $b$

if $\gcd(a, m) = 1; x \equiv a^{-1} \cdot b \pmod{m}$

```
x = (b*pow(a,-1,m) ) %m
```

# 02 Diffie - Hellman Protocol



# 02 Diffie - Hellman Protocol

$$g^a \bmod p = g \cdot g \cdot \ldots \cdot g \ \ (a \text{ times}) \quad (\bmod \ p)$$

Powers in $\mathbb{Z}_p$ have the same properties as powers in $\mathbb{Z}$

$$(g^a)^b \bmod p \equiv (g^b)^a \bmod p \equiv g^{a \cdot b} \bmod p$$

$g^{-a} \quad \bmod \ p =$ inverse of $g^a$ with respect to $p$ (easy to calculate)

$$g^a \cdot g^b \bmod p \equiv g^{a+b} \bmod p$$

$$g^a \cdot g^{-b} \bmod p \equiv g^{a-b} \bmod p$$

$$0^a \bmod p \equiv 0, 1^a \bmod p \equiv 1$$

$$(p-1)^a \bmod p \equiv (-1)^a \bmod p \equiv \pm 1$$

**Could be really useful**

Example with $g = 3, p = 7$

$g^0 \bmod p = 1$
$g^1 \bmod p = 3$
$g^2 \bmod p = 2$
$g^3 \bmod p = 6$
$g^4 \bmod p = 4$
$g^5 \bmod p = 5$
$g^6 \bmod p = 1$
$g^7 \bmod p = 3$
$g^8 \bmod p = 2$
$g^9 \bmod p = 6$

# 02 Diffie - Hellman Protocol

Example with $g = 3, p = 7$

$$
\begin{aligned}
g^0 \bmod p &= 1 \\
g^1 \bmod p &= 3 \\
g^2 \bmod p &= 2 \\
g^3 \bmod p &= 6 \\
g^4 \bmod p &= 4 \\
g^5 \bmod p &= 5 \\
g^6 \bmod p &= 1 \\
g^7 \bmod p &= 3 \\
g^8 \bmod p &= 2 \\
g^9 \bmod p &= 6
\end{aligned}
$$

Example with $g = 2, p = 5$

$$
\begin{aligned}
g^0 \bmod p &= 1 \\
g^1 \bmod p &= 2 \\
g^2 \bmod p &= 4 \\
g^3 \bmod p &= 3 \\
g^4 \bmod p &= 1 \\
g^5 \bmod p &= 2 \\
g^6 \bmod p &= 4 \\
g^7 \bmod p &= 3 \\
g^8 \bmod p &= 1 \\
g^9 \bmod p &= 2
\end{aligned}
$$

If $\gcd(g, p) = 1; g^{p-1} \equiv 1 \pmod{p}$

$g^{(p-1)\cdot b} \bmod p \equiv (g^{p-1})^b \bmod p \equiv 1^b \bmod p \equiv 1 \pmod{p}$

Cute 😻

But what if the Modulus is not Prime

We'll see after with RSA

# DIFFIE-HELLMAN PROTOCOL

# 02 Diffie - Hellman Protocol

**ALICE** **COMMUNICATION BETWEEN A & B** **BOB**





Alice decides a generator $g$ and a modulus $p$ (usually they are standardized)

Alice generates her private key $a \in \{2, 3, \ldots, p-3, p-2\}$

Alice calculates her public key $A \equiv g^a \pmod{p}$ and sends to Bob $(g, p, A)$

Bob generates his private key $b \in \{2, 3, \ldots, p-3, p-2\}$

Bob calculates and sends his public key $B \equiv g^b \pmod{p}$

Alice calculates the shared key as $S_a \equiv B^a \pmod{p}$

Bob calculates the shared key as $S_b \equiv A^b \pmod{p}$

# 02 Diffie - Hellman Protocol

**ALICE** **COMMUNICATION BETWEEN A & B** **BOB**





Alice decides $g = 2, p = 5$

$a = 3$

$A \equiv 2^3 \bmod 5 \equiv 3$

**Sends to Bob**

**Sends to Alice**

$b = 2$

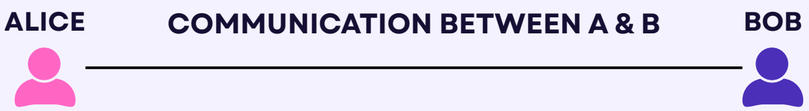
$B = 2^2 \bmod 5 \equiv 4$

Alice calculates the shared key as $S_a = 4^3 \pmod 5 \equiv 4$

Bob calculates the shared key as $S_b = 3^2 \pmod 5 \equiv 4$

**PROVE THAT IT WORKS AND THAT IT'S SAFE**

# 02 Diffie - Hellman Protocol




Alice decides a generator $g$ and a modulus $p$ (usually they are standardized)
Alice generates her private key $a \in \{2, 3, \ldots, p-3, p-2\}$
Alice calculates her public key $A \equiv g^a \pmod{p}$ and sends to Bob $(g, p, A)$

Bob generates his private key $b \in \{2, 3, \ldots, p-3, p-2\}$
Bob calculates and send his public key $B \equiv g^b \pmod{p}$

Alice calculates the shared key as $S_a \equiv B^a \pmod{p}$

Bob calculates the shared key as $S_b \equiv A^b \pmod{p}$

## PROVE THAT IT WORKS

$$S_a \equiv S_b \pmod{p} \implies A^b \equiv B^a \pmod{p}$$

$$\implies (g^a)^b \equiv (g^b)^a \pmod{p}$$

**This is a basic property of powers**

## PROVE THAT IT WORKS AND THAT IT'S SAFE

## PROVE THAT IT'S SAFE

Given $g, p, A \equiv g^a \bmod p$ impracticable to find $a$

**The proof is based on man's ignorance**

Finding $x$ given $c = g^x \bmod p$ is believed to be NP-Intermediate

**We can't prove it though**

**This problem is called the**

## DISCRETE LOG PROBLEM

## 02 Diffie - Hellman Protocol

# DISCRETE LOG PROBLEM

**Reminder:** A problem being NP-Intermediate doesn't mean it can't become easy to solve if we choose bad parameters.

Requirements for $p$ and $g$ (Safe Parameters)

$$0^a \bmod p \equiv 0, 1^a \bmod p \equiv 1$$
$$(p-1)^a \bmod p \equiv -1^a \bmod p \equiv \pm 1$$

Could be really useful

Ensure that $g$ doesn't allow algebraic shortcuts $\implies g \notin \{0, 1, p-1\}$

$p$ must be sufficiently large (today standards $\geq 3072$ bits)

# 02 Diffie - Hellman Protocol

## SO P JUST NEED TO BE LARGE ENOUGH?

What happens if $p-1$ has small factors $\quad g^{(p-1)} \bmod p \equiv (g^{\frac{(p-1)}{2}})^2 \bmod p \equiv 1$

$g \equiv h^{\frac{(p-1)}{2}} \pmod{p}$ is a value such that $g^2 \equiv 1 \pmod{p}$

$g \equiv h^{\frac{(p-1)}{3}} \pmod{p}$ is a value such that $g^3 \equiv 1 \pmod{p}$

$$p = 7, g = 5^{\frac{(p-1)}{2}} \bmod p = 6$$

$$g^0 \bmod p \equiv 1$$
$$g^1 \bmod p \equiv 6$$
$$g^2 \bmod p \equiv 1$$
$$g^3 \bmod p \equiv 6$$
$$g^4 \bmod p \equiv 1$$
$$g^5 \bmod p \equiv 6$$
$$g^6 \bmod p \equiv 1$$

$$p - 1 = 6$$

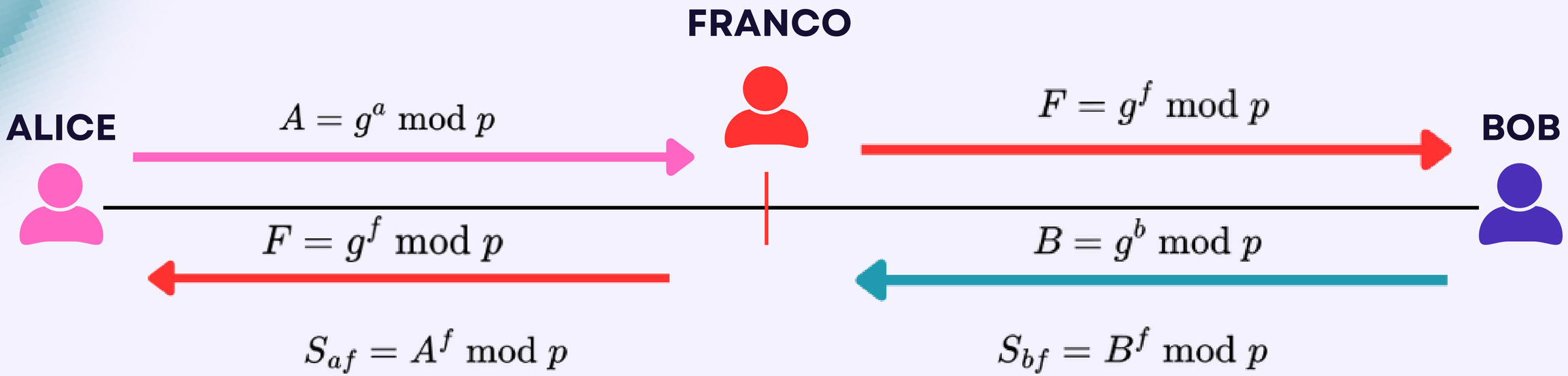
$$p = 7, g = 5^{\frac{(p-1)}{3}} \bmod p = 4$$

$$g^0 \bmod p \equiv 1$$
$$g^1 \bmod p \equiv 4$$
$$g^2 \bmod p \equiv 2$$
$$g^3 \bmod p \equiv 1$$
$$g^4 \bmod p \equiv 4$$
$$g^5 \bmod p \equiv 2$$
$$g^6 \bmod p \equiv 1$$

**WOW WE CAN ARBITRARILY REDUCE THE ORDER OF OUR GENERATORS I WONDER HOW THIS COULD BE USEFUL**

# 02 Diffie - Hellman Protocol

## FRANCO INTERCEPTION




- Franco sends his public key to both the parties.
- Alice and Bob share the key with Franco

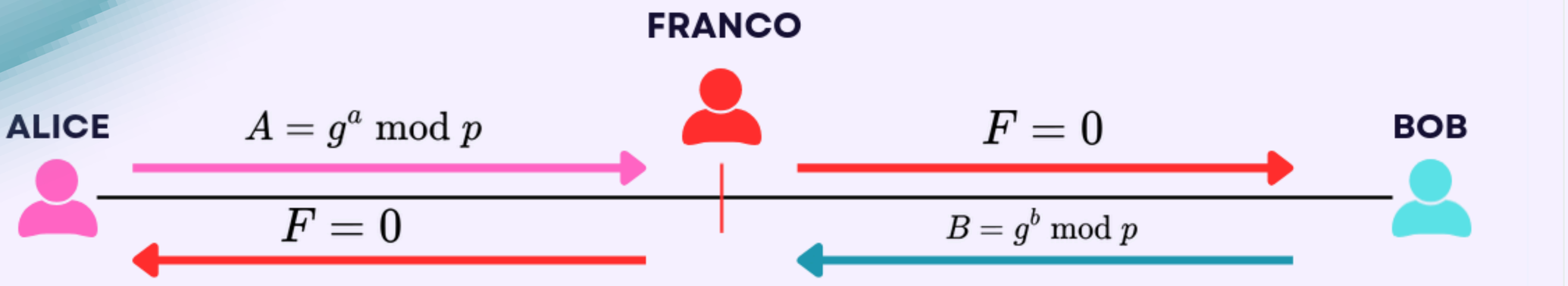
Franco decrypts the messages sent from Alice using $S_{af}$

read them and re-encrypt them with $S_{bf}$ and sends them to Bob

**NO ONE NOTICES**

# 02 Diffie - Hellman Protocol

## LAZY FRANCO




$$S_{af} \equiv S_{bf} \pmod{p} \equiv 0$$

$$S_{af} = F^a \pmod{p} \equiv 0^a \bmod p \equiv 0$$

$$S_{bf} = F^b \pmod{p} \equiv 0^b \bmod p \equiv 0$$

**WE'VE SEEN BEFORE OTHER WAYS TO MAKE PREDICTABLE KEYS**

$$p = 2 \cdot 3 \cdot 7 \cdot \text{big prime} + 1$$

$$g = h^{\frac{(p-1)}{3}} \pmod{p}, g = h^{\frac{(p-1)}{7}} \pmod{p}, g = h^{\frac{(p-1)}{2}} \pmod{p}$$

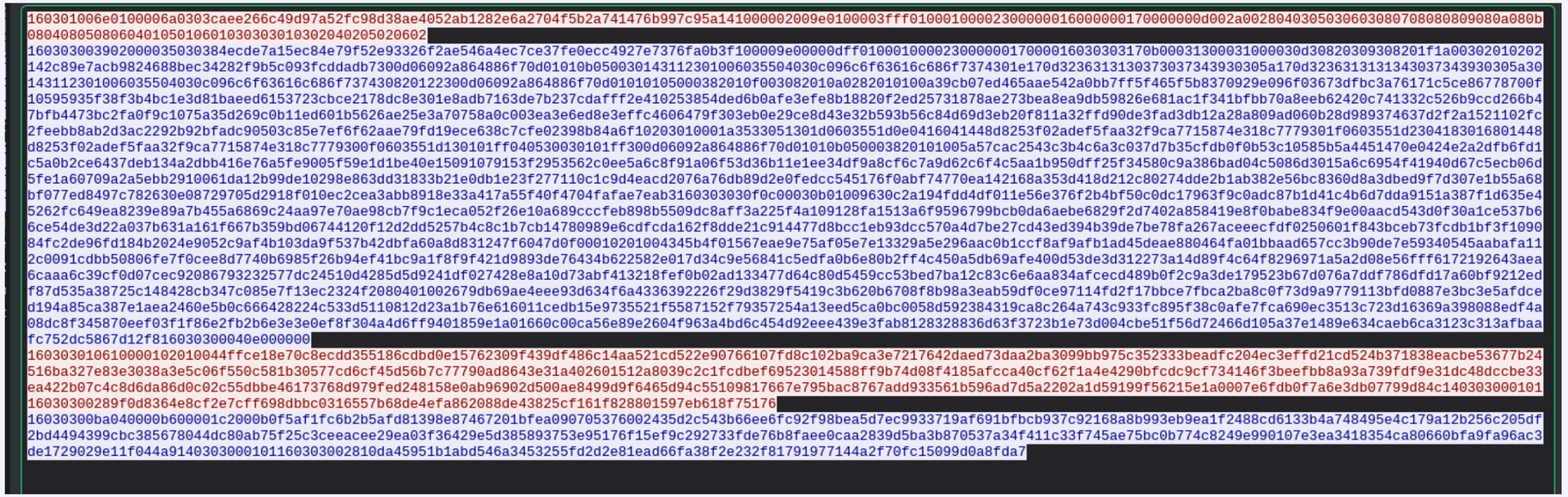
# ENSURING SAFETY IN DIFFIE HELLMAN

Just ensure that $p = 2 \cdot q + 1$ where $q$ is a big prime and make sure $g \notin \{0, 1, p - 1\}$

## SURE, ENCRYPTION KEEPS SECRETS... BUT HOW DO WE KNOW WHO SENT IT AND THAT NO ONE TAMPERED WITH IT ALONG THE WAY?

# 02 Diffie - Hellman Protocol

| No. | Time | Source | Destination | Protocol | Lengtl | Info |
|---|---|---|---|---|---|---|
| 6 | 1794037… | ::1 | ::1 | TLSv1.2 | 1741 | Server Hello, Certificate, Server Key Exchange, Server Hello Done |
| 8 | 1794037… | ::1 | ::1 | TLSv1.2 | 404 | Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message |



# 02 Diffie - Hellman Protocol

```
 sgrenz …/ctf_and_things/Cryptography/randthem0le  main X!?  v3.11.15(#.venv)  sage  09:24 › python main.py packets.txt --full
p: 2048 bits  9630c2a194fdd4df011e56e376f2b4bf50c0dc17963f9c0adc87b1d41c4b6d7dda9151a387f1d635e45262fc649ea8239e89a7b455a6869c24aa97e70ae98cb7f9c1eca052f26e10a689cccfeb898b55
58419e8f0babe834f9e00aacd543d0f30a1ce537b66ce54de3d22a037b631a161f667b359bd06744120f12d2dd5257b4c8c1b7cb14780989e6cdfcda162f8dde21c914477d8bcc1eb93dcc570a4d7be27cd43ed394b39
84b2024e9052c9af4b103da9f537b42dbfa60a8d831247f6047d0f
g: 2
A = server public (Ys): 2048 bits  4345b4f01567eae9e75af05e7e13329a5e296aac0b1ccf8af9afb1ad45deae880464fa01bbaad657cc3b90de7e59340545aabafa112c0091cdbb50806fe7f0cee8d7740b698
fa0b6e80b2ff4c450a5db69afe400d53de3d312273a14d89f4c64f8296971a5a2d08e56fff6172192643aea6caaa6c39cf0d07cec92086793232577dc24510d4285d5d9241df027428e8a10d73abf413218fef0b02ad13
23b67d076a7ddf786dfd17a60bf9212edf87d535a38725c148428cb347c085e7f13ec2324f2
signature algorithm: 0804 - signature length: 256
p prime: True | q = (p-1)/2 prime (safe prime): True | p mod 24: 23
g^q mod p: 1 (g has order q)
A^q mod p: 1 (A in the order-q group)
B = client public (Yc): 2048 bits  44ffce18e70c8ecdd355186cdbd0e15762309f439df486c14aa521cd522e90766107fd8c102ba9ca3e7217642daed73daa2ba3099bb975c352333beadfc204ec3effd21cd52
f45d56b7c77790ad8643e31a402601512a8039c2c1fcdbef69523014588ff9b74d08f4185afcca40cf62f1a4e4290bfcdc9cf734146f3beefbb8a93a739fdf9e31dc48dccbe33ea422b07c4c8d6da86d0c02c55dbbe467
5bac8767add933561b596ad7d5a2202a1d59199f56215e1a0007e6fdb0f7a6e3db07799d84c
B^q mod p: 1
```

TY CLAUDE FOR THE PARSER

## 02 Diffie - Hellman Protocol

# TIME FOR CHALLENGES

## 02 Diffie - Hellman Protocol

# HOW DO WE ENSURE THAT THE MESSAGES AREN'T BEING MODIFIED ??

# DIGITAL SIGNATURES

## 03 RSA and Digital Signatures ( DSA )

We sign messages so that if anyone modifies them, the signature stops being valid and we know not to trust them.

# RSA - PROTOCOL

RSA CAN BE USED TO ENCRYPT DATA AND TO SIGN IT

# 01 Introduction to Asymmetric Cryptography

## THE IDEA

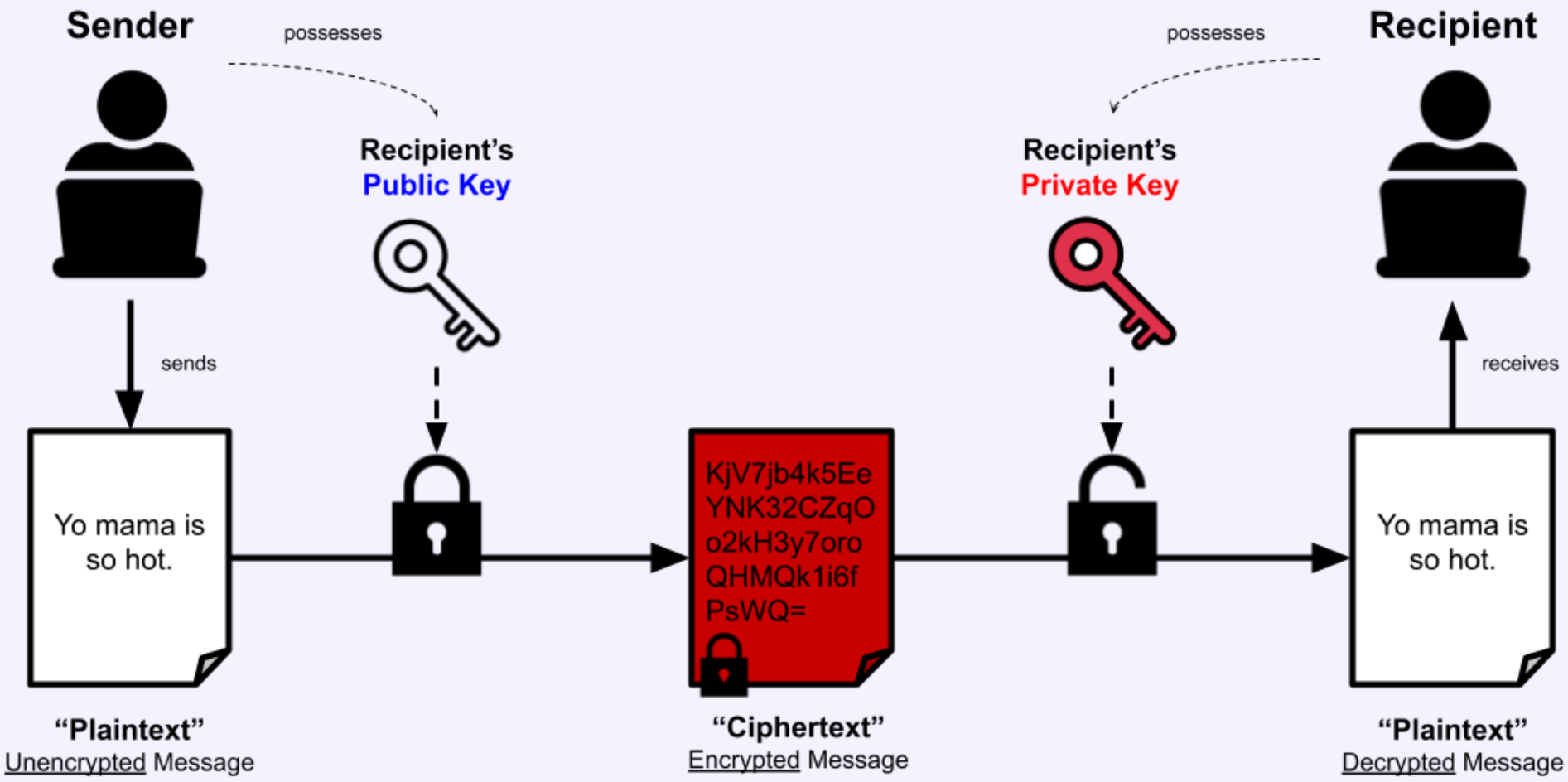
### We want to create a computationally hard problem

### The Mathematical Foundation

**one-way function** is a mathematical operation that is easy to compute in one direction, **but computationally infeasible to invert given only the output.** In asymmetric cryptography, this guarantees that generating a public key from a private value or encrypting a message is fast, while deriving the original secret from the public data would require an impractical amount of time and computing power.

**A trapdoor function** is a special type of one-way function that contains a secret "shortcut" (the trapdoor), anyone who possesses the trapdoor—which corresponds to the private key—can easily invert the process and decrypt the data in milliseconds.

# How Asymmetric Encryption Works (Classic Example)



# Euler's Totient Function

If $\gcd(g,p) = 1; g^{p-1} \equiv 1 \pmod{p}$ If $\gcd(g,n) = 1; g^{\varphi(n)} \equiv 1 \pmod{n}$

$$n = p_1^{k_1} \cdot p_2^{k_2} \cdot p_3^{k_3} \dots$$

$$\varphi(n) = p_1^{k_1-1} \cdot (p_1 - 1) \cdot p_2^{k_2-1} \cdot (p_2 - 1) \cdot p_3^{k_3-1} \cdot (p_3 - 1) \dots$$

$$\text{If } n \text{ is prime} \implies \varphi(n) = n - 1$$

$$\text{If } n \text{ is the product of two primes } (p,q) \implies \varphi(n) = (p-1) \cdot (q-1)$$

# 03 RSA and Digital Signatures ( DSA )

## RSA - ENCRYPTION

$m =$ il messaggio interpretato come un intero big-endian, con $0 \leq m < N$

$e =$ Public exponent

```
m = Crypto.Util.number.bytes_to_long(message)
```

$N = p \cdot q$, Where $p$ and $q$ are two large primes

Public-key $= (e, N)$

$$\text{ct} = m^e \bmod N$$

```
ct = pow(m,e,N)
```

```
ciphertext = Crypto.Util.number.long_to_bytes(ct)
```

**IF WE WANT TO SEND BYTES TO A SERVER WE USUALLY ENCODE IT IN HEX FORMAT**

```
ciphertext_to_send = ciphertext.hex()
```

```
ciphertext_recieved = bytes.fromhex(ciphertext_to_send)
```

# 03 RSA and Digital Signatures ( DSA )

## RSA - DECRYPTION

$$\text{ct} = m^e \bmod N$$

If $\gcd(g, n) = 1; g^{\varphi(n)} \equiv 1 \pmod{n} \implies g^{\varphi(n)+1} \equiv g \pmod{n} \implies g^{k(\varphi(n))+1} \equiv g \pmod{n}$

We want to find an exponent $d$ such that $ct^d \bmod N \equiv m$

We need to find $d$ such that $d \cdot e = 1 + k \cdot \varphi(N) \equiv 1 \pmod{\varphi(N)}$

```
phi_N = (p-1) * (q-1)
```

```
d = pow(e,-1,phi_N)
```

$$m \equiv \text{ciphertext}^d \pmod{N} \equiv (m^e)^d \pmod{N} \equiv m^1 \pmod{N}$$

$e$ has to be coprime with $\varphi(N)$

```
m = pow(ct,d,N)
```

$$\text{Private-key} = (d, \varphi(N))$$

```
message = long_to_bytes(m)
```

# 03 RSA and Digital Signatures ( DSA )

ALICE

ALICE WANTS TO SEND A MESSAGE TO BOB

BOB'S SERVER





Bob generates 2 big primes$(p, q)$ and calculates $N = p \cdot q$

Bob calculates $\varphi(N)$

Bob looks for prime $e$, such that $\gcd(e, \varphi(N)) = 1)$

Bob sends $(e, N)$

Alice calculates and sends $\text{ct} = m^e \mod N$

Bob calculates $d$, solving $d \cdot e \equiv 1 \mod \varphi(N)$

Bob reads m, $\text{m} = \text{ct}^d \mod N$

PROVE THAT IT WORKS AND THAT IT'S SAFE

# PROVE THAT IT WORKS AND THAT IT'S SAFE

## PROVE THAT IT WORKS

$$m \equiv \text{ciphertext}^d \pmod{N} \equiv (m^e)^d \pmod{N} \equiv m^1 \pmod{N}$$

## PROVE THAT IT'S SAFE

To calculate $\varphi(N) = (p-1)\cdot(q-1)$, I need to know $p$ and $q$

Knowing only $N$ I don't have an algorithm to compute $\varphi(N)$

So the complexity of RSA is purely based on the difficulty of factoring $N$

**Which is a suspected NP-Intermediate Problem**

# 03 RSA and Digital Signatures ( DSA )

## CLASSIC RSA VULNERABITIES

If $m^e < N \implies \text{ct} = m^e$, We can just compute the $e$-th root to find $m$

```
m,exact = gmpy2.iroot(ct,e)
```

If i know a number $R$ that shares a factor with $N, \gcd(N, R) \in \{p, q\}$

```
p = math.gcd(N,R)          q = N // p
```

## HOMOMORPHIC PROPERTY

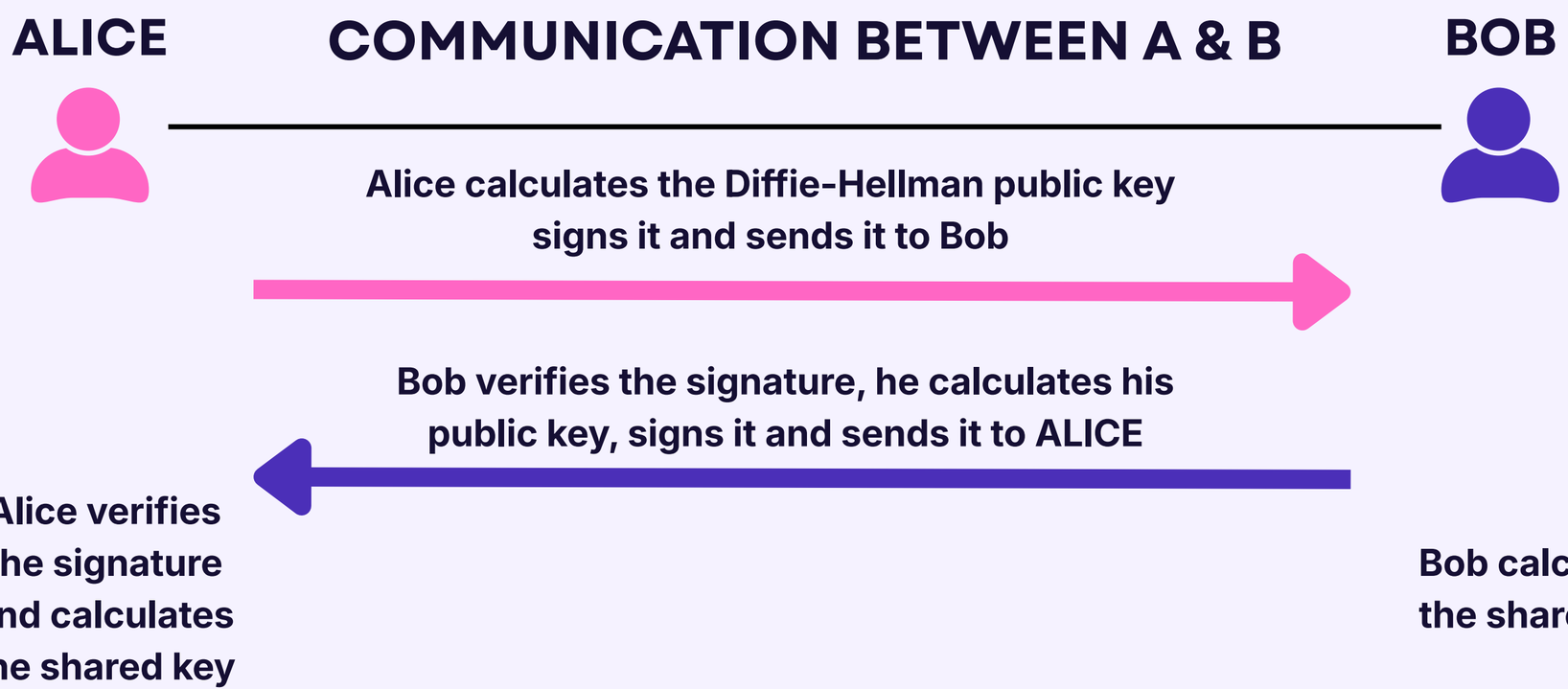
$$m = m_1 \cdot 2; ct = m^e \bmod N \equiv 2^e \cdot m_1^e \bmod N$$

# 03 RSA and Digital Signatures ( DSA )

## A SIGNATURE IS A VALUE THAT CERTIFIES THAT THE MESSAGE HASN'T BEEN TAMPERED WITH

### A SIGNATURE PROTOCOL IS BASED ON TWO FUNCTIONS: SIGNING AND VERIFYING, WE SIGN WITH THE PRIVATE KEY AND VERIFY WITH THE PUBLIC KEY

## Anyone can verify, only the sender can sign



# 03 RSA and Digital Signatures ( DSA )

ALICE — COMMUNICATION BETWEEN A & B — BOB





Alice calculates the Diffie-Hellman public key
signs it and sends it to Bob

Alice generates 2 big primes$(p, q)$ and calculates $N = p \cdot q$

Alice calculates $\varphi(N)$

Alice looks for prime $e$, such that $\gcd(e, \varphi(N)) = 1)$

Alice calculates the signature $s$

Alice sends $(s, e, N)$

**RSA - SIGNING**

```
s = pow(hash(message),d,N)
```

**HASHING IS A ONE-WAY FUNCTION NO ONE CAN REVERT AN HASH BUT ANYBODY CAN CALCULATE A MESSAGE'S HASH**

**HASH(MESSAGE) IS JUST A WAY TO SAY THAT THE HASH FUNCTION CAN BE WHATEVER COLLISION RESISTANT SAFE HASH USUALLY SHA256**

# 03 RSA and Digital Signatures ( DSA )




## RSA - VERIFYING

```
s = pow(hash(message),d,N)
```

```
if hash(message) == pow(s,e,N):
    print("Valid Signature")
```

## PROOF:

$$s = H(m)^d \bmod N;\ s^e \bmod N \equiv H(m)^{d \cdot e} \bmod N \equiv H(m) \bmod N$$

## 03 RSA and Digital Signatures ( DSA )

# VULNERABILITIES ??

## ALWAYS HASH THE MESSAGES

$$m_1 \neq m_2; H(m_1) \equiv H(m_2) \pmod{N} \text{ infeasible}$$

$$m_1 \neq m_2; m_2 \equiv m_1 \pmod{N}; m_2 = m_1 + N$$

## HOMOMORPHIC PROPERTY

$$m = m_1 \cdot 2; ct = m^e \bmod N \equiv 2^e \cdot m_1^e \bmod N$$

# 03 RSA and Digital Signatures ( DSA )

## DSA

$g \equiv h^{\frac{(p-1)}{2}} \pmod{p}$ is a value such that $g^2 \equiv 1 \pmod{p}$

$g \equiv h^{\frac{(p-1)}{3}} \pmod{p}$ is a value such that $g^3 \equiv 1 \pmod{p}$

$g \equiv h^{\frac{(p-1)}{q}} \pmod{p}$, we are forcing the order to be $q$ $q$ is a big factor of $p-1$

$x =$ Private Key $y \equiv g^x \bmod p =$ Public Key

$k =$ Nonce, random private Integer, should never be reused

## DSA - SIGNING

$r = (g^k \bmod p) \bmod q$

$s = k^{-1}(H(m) + x \cdot r) \bmod q$

**No python implementation, you'll have to do it in the challenges**

To Verify Bob needs $y, m, g, r, s, p, q$

## 03 RSA and Digital Signatures ( DSA )

# DSA - VERIFYING

**Just some boring equations all the main vulnerabilities are in the signing phase dont worry**

# VULNERABILITIESS???

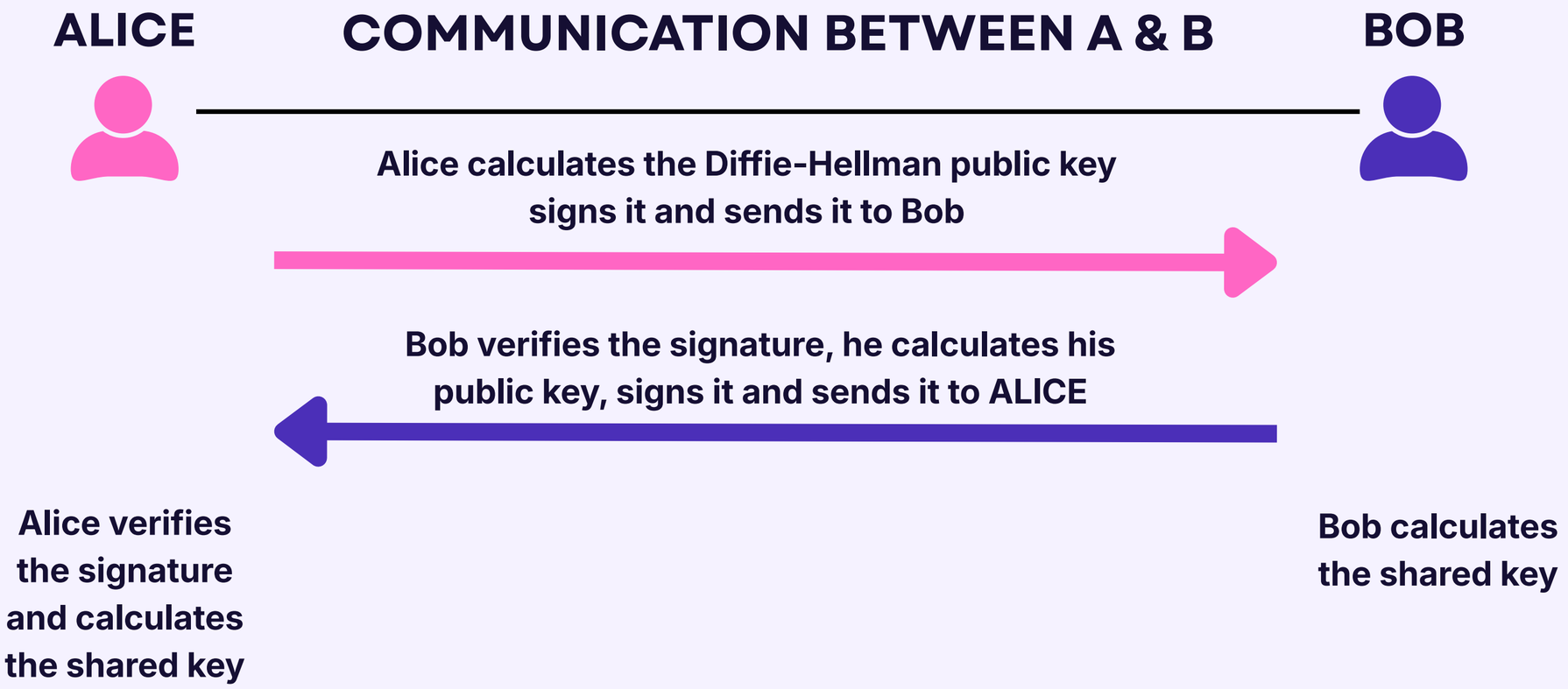
$k =$ Nonce, What could happen if you reuse more times something called nonce

**Even if the nonces are related DSA is still broken**

$$g \equiv h^{\frac{(p-1)}{q}} \pmod{p},$$ we are forcing the order to be $q$

**Maybe if we can choose parameters we can lower the order**

# 03 RSA and Digital Signatures ( DSA )




**We still have a big problem tho, we can verify the correctness of the signature, but how do we verify that the sender is Bob and not FRANCO**

# How does TLS fix the problem?

ALICE COMMUNICATION BETWEEN A & B BOB'S SERVER

## CERTIFICATES




Bob's Identity (domain)
Bob's Public Key
Validity period
CA digital signature

A Certification Authority (CA) is a trusted third party that verifies who owns a public key and vouches for it by signing a certificate. Clients only need to trust the CA, not every individual peer.

# ARE WE STILL SUPER SAFE ??

**maybe.**

## IF A CA IS COMPROMISED, SOMEONE CAN FORGE CERTIFICATES THAT BROWSERS ACCEPT WITHOUT ANY WARNING. TRUST IS DECIDED BY THE ROOT PROGRAMS OF OS AND BROWSER VENDORS, NOT BY THE USER.

PCQuest

**eMudhra's weak certificate keys expose gaps in browser root program oversight**

14 days ago | Mudhra's weak certificate keys went unnoticed for 18 months. Why did browser security checks miss a flaw that took just 27 minutes to detect?




**The digital signatures that we find in certificates are made using the CA's private key, safe to say that using 4 as a private key could be a little security concern**

**For 18 months anybody could have forged any certificates of any domain and if on the same connection perform MITM**

# Did Not Connect: Potential Security Issue

Firefox detected a potential security threat and did not continue to [redacted] because this website requires a secure connection.

**What can you do about it?**

[redacted] has a security policy called HTTP Strict Transport Security (HSTS), which means that Firefox can only connect to it securely. You can't add an exception to visit this site.

The issue is most likely with the website, and there is nothing you can do to resolve it.

If you are on a corporate network or using antivirus software, you can reach out to the support teams for assistance. You can also notify the website's administrator about the problem.

Learn more...

Go Back | Advanced...

Someone could be trying to impersonate the site and you should not continue.

Websites prove their identity via certificates. Firefox does not trust www.coursera.org because its certificate issuer is unknown, the certificate is self-signed, or the server is not sending the correct intermediate certificates.

Error code: SEC_ERROR_UNKNOWN_ISSUER

View Certificate

Go Back

## Certificate

[redacted] | Workshop Rogue CA

**Subject Name**

Common Name [redacted]

**Issuer Name**

Common Name Workshop Rogue CA
Organization MITM Workshop

**Validity**

Not Before Sun, 23 Aug 2026 00:00:00 GMT
Not After Mon, 08 Mar 2027 23:59:59 GMT

**Subject Alt Names**

DNS Name [redacted]
DNS Name

**Public Key Info**

Algorithm RSA
Key Size 1024
Exponent 65537
Modulus C3:6F:B3:17:A3:10:58:EE:4F:38:A7:03:27:18:D7:A6:E8:89:C9:07:B5:82:B7:3...

## Certificate

[redacted] | Workshop Rogue CA

**Subject Name**

Common Name Workshop Rogue CA
Organization MITM Workshop

**Issuer Name**

Common Name Workshop Rogue CA
Organization MITM Workshop

**Validity**

Not Before Mon, 02 Nov 2026 11:28:55 GMT
Not After Sat, 01 Nov 2031 11:28:55 GMT

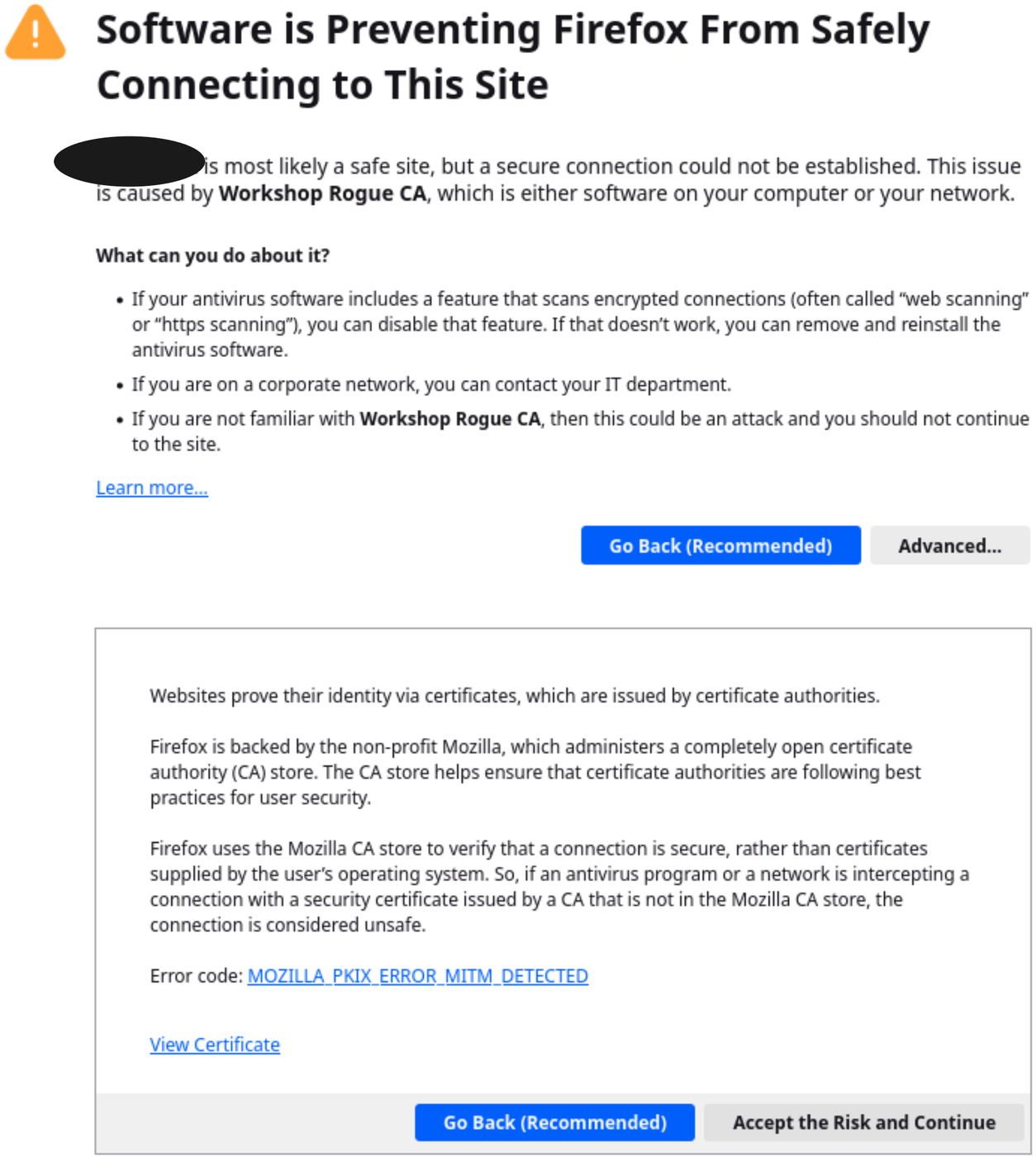
**Public Key Info**

Algorithm RSA
Key Size 4096
Exponent 65537
Modulus 86:FB:37:9E:5B:23:40:33:BF:52:52:75:B4:2F:3D:6E:E6:8E:66:56:DD:34:ED:4...

**Miscellaneous**

Serial Number 09:0C:61:49:E4:0E:AD:CB:D2:0C:89:79:DA:93:84:D8:8A:78:B9:33
Signature Algorithm SHA-256 with RSA Encryption
Version 3
Download PEM (cert) PEM (chain)

# ON FIRST CONNECTION



## Software is Preventing Firefox From Safely Connecting to This Site

[redacted] is most likely a safe site, but a secure connection could not be established. This issue is caused by **Workshop Rogue CA**, which is either software on your computer or your network.

**What can you do about it?**

- If your antivirus software includes a feature that scans encrypted connections (often called "web scanning" or "https scanning"), you can disable that feature. If that doesn't work, you can remove and reinstall the antivirus software.
- If you are on a corporate network, you can contact your IT department.
- If you are not familiar with **Workshop Rogue CA**, then this could be an attack and you should not continue to the site.

Learn more...

Go Back (Recommended) | Advanced...

Websites prove their identity via certificates, which are issued by certificate authorities.

Firefox is backed by the non-profit Mozilla, which administers a completely open certificate authority (CA) store. The CA store helps ensure that certificate authorities are following best practices for user security.

Firefox uses the Mozilla CA store to verify that a connection is secure, rather than certificates supplied by the user's operating system. So, if an antivirus program or a network is intercepting a connection with a security certificate issued by a CA that is not in the Mozilla CA store, the connection is considered unsafe.

Error code: MOZILLA_PKIX_ERROR_MITM_DETECTED

View Certificate

Go Back (Recommended) | Accept the Risk and Continue




github.com

# ⚠ FAKE PAGE ⚠

MITM WORKSHOP DEMONSTRATION

This page loaded with a **valid, trusted certificate** for this domain.

Your browser showed no warning — because a rogue CA was added to the trust store.

No real traffic to the actual site was touched.

---

`rogue CA chain → trusted · nothing real was intercepted`

# CHALLENGESSS

# TIME

# APPENDIX

# 01 Introduction to Asymmetric Cryptography

## The ADDRESS RESOLUTION PROTOCOL (ARP)

**This is the protocol which is used from a device connected to the network to map an IP to its MAC address**

**It is stateless and without authentication 💀💀**

**It means that we can just replicate the packet that the router usually sends and the target is going to believe us**

**This technique is called ARP SPOOFING**

# 05 Boring Math

$$d = \gcd(a, m) \qquad a \cdot x_0 + m \cdot x_1 = b$$

if $d$ divides $b$ we can rewrite the equation like this: $d \cdot a' \cdot x_0 + d \cdot m' \cdot x_1 = d \cdot b'$

**Trivially** $a' = \frac{a}{d}\ m' = \frac{m}{d}\ b' = \frac{b}{d}$ $\qquad a' \cdot x_0 + m' \cdot x_1 = b'$

if $b' = 1$ easy

if $b' \neq 1$?

we solve for $b' = 1$ $\qquad a' \cdot x_{01} + m' \cdot x_{11} = 1$

$$x_0 = x_{01} \cdot b'$$
$$x_1 = x_{11} \cdot b'$$
$$(a' \cdot x_{01} + m' \cdot x_{11}) \cdot b' = b'$$

# 05 Boring Math

$$d = \gcd(a, m) \qquad a \cdot x_0 + m \cdot x_1 = b$$

if $d$ divides $b$ we can rewrite the equation like this: $d \cdot a' \cdot x_0 + d \cdot m' \cdot x_1 = d \cdot b'$

$$\textbf{Trivially } a' = \frac{a}{d}\ m' = \frac{m}{d}\ b' = \frac{b}{d} \qquad a' \cdot x_0 + m' \cdot x_1 = b'$$

if $d$ doesn't divide $b$ we can't do the trick $\implies$ **No Solutions**

The real question is how many solution does $a \cdot x_0 + m \cdot x_1 = b$ have ??

**Exactly d solutions mod m** $x, x + m$ are the same

Given $x_0$ what are the other solutions?

**Figure it out in the challenges**



# 02 Diffie - Hellman Protocol

Example with $g = 3, p = 7$

$$
\begin{aligned}
g^0 \bmod p &= 1 \\
g^1 \bmod p &= 3 \\
g^2 \bmod p &= 2 \\
g^3 \bmod p &= 6 \\
g^4 \bmod p &= 4 \\
g^5 \bmod p &= 5 \\
g^6 \bmod p &= 1 \\
g^7 \bmod p &= 3 \\
g^8 \bmod p &= 2 \\
g^9 \bmod p &= 6
\end{aligned}
$$

Example with $g = 2, p = 5$

$$
\begin{aligned}
g^0 \bmod p &= 1 \\
g^1 \bmod p &= 2 \\
g^2 \bmod p &= 4 \\
g^3 \bmod p &= 3 \\
g^4 \bmod p &= 1 \\
g^5 \bmod p &= 2 \\
g^6 \bmod p &= 4 \\
g^7 \bmod p &= 3 \\
g^8 \bmod p &= 1 \\
g^9 \bmod p &= 2
\end{aligned}
$$

If $\gcd(g, p) = 1; g^{p-1} \equiv 1 \pmod{p}$

$g^{(p-1)\cdot b} \bmod p \equiv (g^{p-1})^b \bmod p \equiv 1^b \bmod p \equiv 1 \pmod{p}$

# 02 Diffie - Hellman Protocol

## ORDER

If $\gcd(g, p) = 1; g^{p-1} \equiv 1 \pmod{p}$

If $p$ is odd prime $> 3 \implies p - 1$ is not

2 always divides $p - 1$ $\quad p - 1 = 2^{k_0} \cdot c_1^{k_1} \cdot c_2^{k_2} \cdot c_3^{k_3} \ldots$ where $c_i$ is prime

For simplicity let's say that $p - 1 = 2 \cdot q,$ where $q$ is prime

$g^{p-1} \bmod p \equiv (g^{\frac{(p-1)}{2}})^2 \bmod p \equiv (g^q)^2 \bmod p \equiv 1$

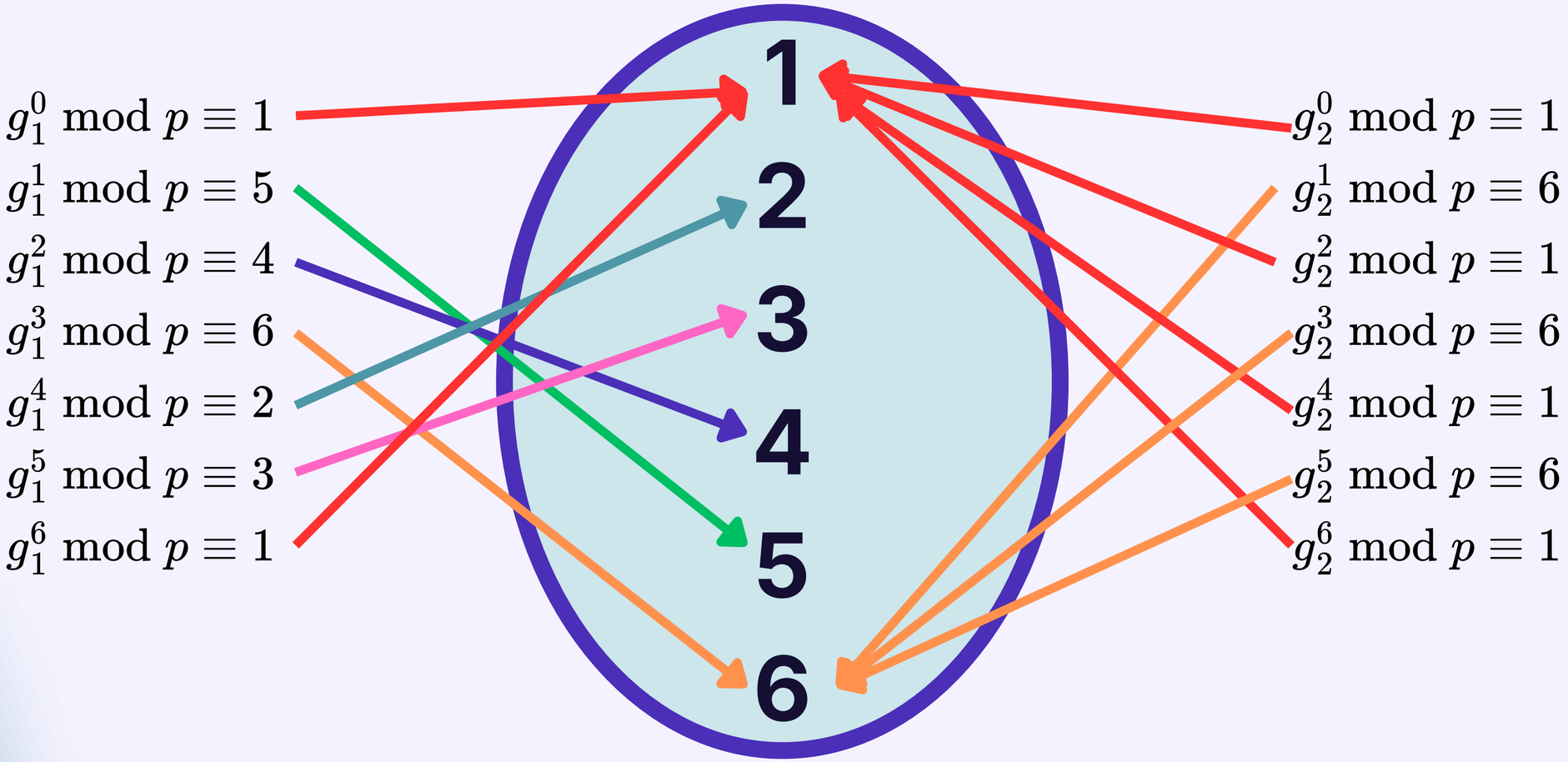
**REMINDER:** We can do this only if 2 divides p-1

I think this can show something interesting

# 02 Diffie - Hellman Protocol

## ORDER

$$p = 7, g_1 = 5, g_2 = 5^{\frac{(p-1)}{2}}$$



# THANK YOU

*By Griin*

Contacts:

- Discord: Griin
- Portfolio: https://griin.pages.dev/